

Chapitre III — Réalisation & Résultats

III.1 — Introduction

Les chapitres précédents ont posé le cadre réglementaire, les objectifs du projet EmissionsIQ et les choix de conception de l'architecture globale. Le présent chapitre franchit l'étape suivante : la **mise en œuvre concrète** de chaque couche du système. On y décrit, dans l'ordre du flux de données, la station IoT physique et son firmware ESP32, le backend Node.js d'ingestion et de calcul, le module d'intelligence artificielle Python, l'interface web React, puis les campagnes de tests et de validation qui confirment le comportement attendu en conditions réelles ou simulées.

La rédaction suit trois principes de concision : des **extraits de code ciblés** (15 à 25 lignes maximum, code complet en annexe), des **captures d'écran annotées** là où une figure vaut deux pages de prose, et des **tableaux de résultats** pour les métriques et scénarios de test. L'objectif est de montrer que les choix du Chapitre II ont produit un système opérationnel, vérifiable et conforme aux exigences du Décret gouvernemental n° 2018-928.

III.2 — Réalisation de la station IoT physique

III.2.1 — Schéma de câblage complet

La station prototype repose sur un **ESP32 DevKit V1** (double cœur Xtensa LX6, WiFi 802.11 b/g/n intégré). Six capteurs couvrent les polluants réglementés et les grandeurs auxiliaires nécessaires au module IA (température, humidité). Le câblage est centralisé dans `pins.h` ; le schéma électrique annoté est disponible dans `documentation/SCHEMA_ELECTRONIQUE.svg`.

Tableau III.1 — Interfaces capteurs / ESP32

Capteur	Polluant / mesure	Interface	Broches ESP32	Alimentation	Particularité
MH-Z19B	CO ₂ (ppm)	UART Serial2 @ 9600	RX=16, TX=17	5 V	Convertisseur de niveau 5 V ↔ 3,3 V obligatoire
SDS011	PM2,5 / PM10 (µg/m ³)	UART Serial1 @ 9600	RX=18, TX=19	5 V	Mode « query » ; level shifter
SGP30	COV / TVOC (mg/Nm ³)	I ² C @ 0x58	SDA=21, SCL=22	3,3 V	Pull-ups 4,7 kΩ sur SDA/SCL
DHT22	Température / humidité	1-wire	DATA=4	3,3 V	Pull-up 10 kΩ sur DATA
MQ-136	SO ₂ (mg/Nm ³)	ADC1 + chauffage	ADC=33, HEATER=26	5 V	Diviseur 10 kΩ / 20 kΩ ; transistor 2N2222
MQ-131	NO _x (mg/Nm ³)	ADC1 + chauffage	ADC=34, HEATER=27	5 V	GPIO34 input-only ; même chaîne analogique

Contraintes électriques retenues :

- Alimentation secteur **220 V AC** → **5 V DC / 3 A**, fusible 1 A en amont, condensateurs de filtrage 100 µF + 100 nF sur chaque rail.
- Régulateur **LM2596** (5 V → 3,3 V) et module **MB102** pour la distribution.
- **Masse commune (GND)** entre tous les composants.
- **ADC1 uniquement** (GPIO 32–39) : l'ADC2 est incompatible avec le WiFi actif sur l'ESP32.
- Tension maximale en entrée ADC : **3,3 V** (diviseurs résistifs sur les sorties MQ à 5 V).

[Figure III.1 — À insérer : documentation/SCHEMA_ELECTRONIQUE.svg exporté en PNG]

Annotations suggérées : flèches vers level shifters UART, encadré « diviseur 10k/20k → ADC1 », bus I²C avec pull-ups, alimentation 5 V/3 A.

III.2.2 — Réalisation physique

Le prototype est monté dans un **boîtier IP54** en polycarbonate, percé pour les entrées d'air des capteurs SDS011 et MH-Z19B (échantillonnage passif). Les capteurs MOS (MQ-136, MQ-131) sont positionnés en zone de flux d'air

représentatif de la zone industrielle surveillée. Un préchauffage de **3 minutes** est imposé au démarrage avant toute lecture MQ fiable.

L'alimentation 5 V / 3 A alimente simultanément l'ESP32 (via VIN), les capteurs UART 5 V et les filaments MQ (~150–200 mA chacun). Des condensateurs de découplage (100 µF électrolytique + 100 nF céramique) sont placés au plus près de chaque capteur numérique pour limiter les perturbations lors des impulsions de chauffage MOS.

[Figure III.2 — À insérer : photographie de la station montée]

Annotations : boîtier, ESP32, MH-Z19B, SDS011, bus de distribution 5 V, câble Ethernet/WiFi.

III.2.3 — Firmware ESP32 — implémentation

Le firmware PlatformIO (`iot/iot/`) utilise le framework **Arduino** sur la plateforme **espressif32**, board **esp32doit-devkit-v1**. Les bibliothèques principales sont : PubSubClient ^2.8, ArduinoJson ^7.2, MH-Z19 ^1.5.4, Adafruit SGP30 ^2.0.2, DHT sensor library ^1.4.6.

Boucle principale — scheduler non bloquant

La fonction `loop()` vérifie d'abord la connectivité, puis déclenche les publications selon quatre intervalles indépendants :

Famille	Intervalle	Constante
CO ₂ (MH-Z19B)	10 s	INTERVAL_CO2_MS
Particules (SDS011)	15 s	INTERVAL_PM_MS
Gaz MOS + SGP30	30 s	INTERVAL_GAS_MS
Environnement (DHT22)	10 s	INTERVAL_ENV_MS

```
// main.cpp – boucle principale (extrait)
void loop() {
  if (WiFi.status() != WL_CONNECTED) connectWiFi();
  if (!mqtt.connected()) connectMqtt();
  mqtt.loop();

  const unsigned long now = millis();
  if (now - lastCo2Ms >= INTERVAL_CO2_MS) { lastCo2Ms = now; publishCo2(); }
  if (now - lastPmMs >= INTERVAL_PM_MS) { lastPmMs = now; publishParticulates(); }
  if (now - lastGasMs >= INTERVAL_GAS_MS) { lastGasMs = now; publishGases(); }
  if (now - lastEnvMs >= INTERVAL_ENV_MS) { lastEnvMs = now; publishEnvironment(); }
  delay(50);
}
```

Reconnexion WiFi / MQTT

Au démarrage et à chaque perte de lien, `connectWiFi()` boucle jusqu'à association réussie (500 ms entre tentatives). `connectMqtt()` retente toutes les **3 secondes** avec un buffer de **512 octets**. L'horodatage repose sur **NTP** (`pool.ntp.org`) pour produire des timestamps ISO 8601 UTC.

Publication MQTT et format JSON

Chaque mesure est publiée sur le topic `emissions/<NODE_ZONE>/<sensorType>` en QoS 0 :

```
// main.cpp - publishReading() (extrait)
static void publishReading(const char *sensorType, const char *model,
                          const char *unit, float value, float rawValue,
                          const char *level, bool isValid) {
    const String topic = String("emissions/") + NODE_ZONE + "/" + sensorType;
    JsonDocument doc;
    doc["sensorType"] = sensorType;
    doc["model"]      = model;
    doc["zone"]       = NODE_ZONE;
    doc["nodeName"]   = NODE_NAME;
    doc["value"]       = value;
    doc["rawValue"]    = rawValue;
    doc["unit"]        = unit;
    doc["level"]       = level;
    doc["timestamp"]   = isoTimestamp();
    doc["isValid"]     = isValid;
    doc["rssi"]        = WiFi.RSSI();
    doc["battery"]     = nullptr;
    char payload[384];
    serializeJson(doc, payload, sizeof(payload));
    mqtt.publish(topic.c_str(), payload, false);
}
```

Exemple de payload publié :

```
{
  "sensorType": "NOX",
  "model": "MQ-131",
  "zone": "Zone-A",
  "nodeName": "Station-Sfax-01",
  "value": 425.80,
  "rawValue": 1.247,
  "unit": "mg/Nm³",
  "level": "warning",
  "timestamp": "2026-06-15T14:00:05Z",
  "isValid": true,
  "rssi": -58,
  "battery": null
}
```

Le champ `level` est indicatif (seuils locaux firmware) ; le backend recalcule les alertes officielles à partir des VLE MongoDB.

III.3 — Implémentation Backend

III.3.1 — Environnement de développement

Stack et versions (`backend/package.json`) :

Composant	Version
Node.js	≥ 20 (requis par Mongoose 9)
Express	5.2.1
Mongoose	9.3.1
Driver MongoDB	7.1.0 (transitif)
mqtt (client)	5.15.1
node-cron	3.0.3
Serveur MongoDB	6+ (base <code>pollution_db</code>)

Structure des dossiers :

```

backend/
├── server.js           # Point d'entrée (port 5000)
├── config/            # db.js, jwt.js, ia.js
├── routes/            # 16 routeurs Express
├── controllers/       # Handlers HTTP
├── services/          # Logique métier (22 services)
├── repositories/      # Accès MongoDB
├── models/            # 18 schémas Mongoose
├── middleware/        # auth, RBAC, errors
├── schedulers/kpiScheduler.js
└── tests/

```

L'architecture respecte le pattern **Routes** → **Controllers** → **Services** → **Repositories** défini au Chapitre II.

III.3.2 — Ingestion MQTT → ReadingService

Le service `mqttService.js` s'abonne au topic `emissions/#` (QoS 1). Chaque message déclenche `processMessage()` qui résout le capteur en base, mappe le polluant, puis délègue à `ReadingService.ingestReading()`.

```

// mqttService.js - processMessage() (extrait)
const processMessage = async (topic, payload) => {
  const data = JSON.parse(payload.toString());
  const sensor = await resolveSensorForMessage(data, topic);
  if (!sensor) return;

  let resolvedPolluant = await Polluant.findOne({ code: data.sensorType }).lean();
  if (!resolvedPolluant) return;

  const readingPayload = {
    sensorId: sensor._id.toString(),
    pollutantId: resolvedPolluant._id.toString(),
    nodeId: sensor.sensorNodeId?.toString() || null,
    value: data.value,
    unit: data.unit,
    rawValue: data.rawValue || data.value,
    isValid: data.isValid !== false,
    timestamp: data.timestamp || new Date(),
  };
  await readingService.ingestReading(readingPayload);
};

```

`ingestReading()` valide la plausibilité physique ($value \leq 10 \times VLE$), persiste le document `Reading`, puis appelle `checkAndCreateAlert()` si `isValid === true`.

III.3.3 — AlertService — implémentation

La logique de sévérité et le **cache in-memory** `_activeAlerts` (Map) résident dans `ReadingService.js`. `AlertService.js` gère le cycle de vie côté API (acquittement, escalade manuelle, statistiques).

Règles de sévérité :

Condition	Sévérité
<code>value > VLE × 1,5</code>	Critical
<code>value > VLE</code>	High
<code>value > warningThreshold</code> (80 % VLE)	Warning
Sinon	Aucune alerte (auto-résolution si alerte ouverte)

```
// ReadingService.js - checkAndCreateAlert() (extrait)
let severity = null;
if (readingValue > pollutant.regulatoryLimit * 1.5) {
  severity = alert_severity.Critical;
} else if (readingValue > pollutant.regulatoryLimit) {
  severity = alert_severity.High;
} else if (readingValue > pollutant.warningThreshold) {
  severity = alert_severity.Warning;
}

const sensorKey = `${String(reading.sensorId)}:${String(reading.PollutantId)}`;
const cached = this._activeAlerts.get(sensorKey);

if (!severity && cached) {
  await alertRepository.autoResolve(cached.alertId, "Valeur revenue dans les limites");
  this._activeAlerts.delete(sensorKey);
  return null;
}
// ... création, mise à jour ou déduplication (fenêtre 30 s)
```

Au démarrage, `_warmUpCache()` recharge toutes les alertes ouvertes (`resolvedAt: null`) depuis MongoDB pour garantir la continuité après redémarrage.

III.3.4 — KPIService — implémentation

KPI TD (Taux de Dépassement) — formule : $TD = (N_{breach} / N_{total}) \times 100$

```
// KPIService.js - calculateTD() (extrait)
const vle = pollutant.regulatoryLimit;
const baseMatch = {
  PollutantId: pollutantId,
  timestamp: { $gte: periodStart, $lte: periodEnd },
  isValid: true,
};
const totalCount = await Reading.countDocuments(baseMatch);
const breachCount = await Reading.countDocuments({ ...baseMatch, value: { $gt: vle } });
const tauxDepassement = totalCount > 0 ? (breachCount / totalCount) * 100 : 0;
return { tauxDepassement: parseFloat(tauxDepassement.toFixed(2)), breachCount, totalCount };
```

Le scheduler **node-cron** (`kpiScheduler.js`) déclenche les agrégations :

Tâche	Expression cron	Action
HOURLY	5 * * * *	Agrégation horaire à H:05
DAILY	10 0 * * *	Agrégation quotidienne à 00:10
WEEKLY	20 0 * * 1	Agrégation hebdomadaire (lundi)
MONTHLY	30 0 1 * *	Agrégation mensuelle (1er du mois)

`AggregationService` appelle `KPIService` pour calculer TD, EMJ, IPE et RCO₂, puis persiste un document `AggregateData` .

III.3.5 — Exemples de documents MongoDB

Document `Reading` :

```
{
  "_id": "674a1b2c3d4e5f6789012345",
  "sensorId": "674a00000000000000000001",
  "PollutantId": "674a00000000000000000002",
  "nodeId": "674a00000000000000000003",
  "value": 135.42,
  "unit": "mg/Nm³",
  "rawValue": 1.18,
  "isValid": true,
  "timestamp": "2026-06-21T14:30:00.000Z",
  "createdAt": "2026-06-21T14:30:00.052Z",
  "updatedAt": "2026-06-21T14:30:00.052Z"
}
```

Document `AggregateData` (agrégation DAILY) :


```
{
  "_id": "674b2c3d4e5f6789012345678",
  "siteId": "674900000000000000000001",
  "zoneId": null,
  "pollutantId": "674a00000000000000000002",
  "sensorNodeId": null,
  "period": "DAILY",
  "periodStart": "2026-06-21T00:00:00.000Z",
  "periodEnd": "2026-06-22T00:00:00.000Z",
  "minValue": 45.2,
  "maxValue": 198.7,
  "avgValue": 92.4,
  "stdDeviation": 12.3,
  "sampleCount": 2880,
  "breachCount": 42,
  "warningCount": 156,
  "tauxDepassement": 1.46,
  "emissionKgDay": 15.9744,
  "score": 0.85,
  "overallScore": null,
  "dataQuality": "GOOD",
  "calculatedAt": "2026-06-22T00:10:00.000Z",
  "calculationDuration": 245
}
```

III.4 — Implémentation du module IA

III.4.1 — Dataset d'entraînement — domain shift et solution

Problème initial. Les premiers essais d'entraînement sur des jeux publics (EPA AQS, UCI Air Quality, Beijing Multi-Site) ont produit un **domain shift** fatal : les distributions d'air ambiant urbain (ppm/ppb, concentrations faibles) ne correspondent pas aux émissions industrielles tunisiennes (mg/Nm³, niveaux ×500 à ×10 000 plus élevés).

Tableau III.2 — Comparaison EPA vs simulateur industriel

Polluant	Plage EPA/UCI	Plage industrielle simulée	Conséquence
NOX	0,02–0,08 ppm	374–680 mg/Nm ³	Skill LSTM –205 %
SOX	0,001–0,03 ppm	187–340 mg/Nm ³	Skill –86 %
PM2,5	5–35 µg/m ³	9,4–17 mg/m ³	Skill –370 %
Isolation Forest	—	—	100 % faux positifs sur profils normaux

Solution retenue : dataset hybride industriel synthétique `industrial_hybrid_tunisian` , généré par `generate_industrial_dataset.py` à partir des profils calibrés de `simulator.js` (6 sites × 90 jours × cadence horaire = **103 680 lignes**). Le générateur injecte 5 % d'anomalies (pics, dérives, incohérences), des corrélations combustion (CO₂ ↔ NOX) et des cycles jour/nuit/weekend.

Tableau III.3 — Résultats comparatifs

Critère	Dataset EPA/UCI	Dataset hybride industriel
Taille	~179 000 lignes, 25 sites	103 680 lignes, 6 sites
Faux positifs IF (profils normaux)	100 %	0 %
Skill LSTM global	–16,4 %	+6,65 %
Détection anomalies IF (test)	Non fiable	5,5 % (≈ contamination 5 %)
Représentativité tunisienne	Faible	Calibrée sur VLE Décret 2018-928

III.4.2 — Entraînement Isolation Forest

Pipeline notebook 04 :

1. Pivot long → wide (6 features : NOX, SOX, PM25, PM10, CO2, COV)
2. `StandardScaler` sur l'ensemble d'entraînement
3. `IsolationForest(n_estimators=100, contamination=0.05, random_state=42)`
4. Seuil de décision configuré : `score_threshold = -0.20` (`ia/config.py`)

Partition : 9 072 train / 1 944 val / 1 944 test (vecteurs horaires)

Tableau III.4 — Métriques Isolation Forest

Métrique	Valeur
Anomalies détectées (validation)	5,3 %
Anomalies détectées (test)	5,5 %
Faux positifs profils normaux (Four, Broyage)	0 %
Rappel anomalies injectées	100 %
Précision (test injecté)	72,5 %
F1-score	0,84
Latence inférence API	~23 ms

III.4.3 — Entraînement LSTM

Architecture : fenêtre **48 h × 8 features** (6 polluants + température + humidité) → LSTM(64) → LSTM(32) → Dense → horizon **+1 h à +4 h**.

Partitionnement temporel strict : 70 % train / 15 % validation / 15 % test (pas de fuite temporelle).

Critères go/no-go et résultats :

Critère	Seuil	Obtenu	Statut
Skill global	≥ 0,02	0,0665 (+6,65 %)	✓ PASS
Skill CO ₂	≥ 0,05	0,0587	✓ PASS
Skill PM ₁₀	≥ 0,08	0,2414	✓ PASS
MAE ratio +1 h	≤ 1,15	0,945	✓ PASS
LSTM bat persistance	≥ 6/8 polluants	6/8	✓ PASS

Tableau III.5 — Skill par polluant (LSTM vs persistance)

Polluant	Skill	Source inférence
CO ₂	+5,9 %	LSTM
NOX	+6,1 %	LSTM
SOX	+14,1 %	LSTM
PM2,5	+23,7 %	LSTM
PM10	+24,1 %	LSTM
COV	+16,9 %	LSTM
TEMPERATURE	0 %	Persistence
HUMIDITY	-23,8 %	Fallback persistence

Mécanisme fallback (`ia/inference.py`) : pour chaque polluant, si `skill ≤ 0` ou présence dans `fallback_pollutants` , la prédiction retourne la baseline de persistance naïve ; sinon LSTM pur ou blend selon `blend_alpha` .

```
# inference.py - sélection source par polluant (extrait)
skill = float(self.per_pollutant_skill.get(name, 0.0))
if name in self.fallback or skill ≤ 0:
    source = "PERSISTENCE"
    value = pers_v
elif alpha ≥ 1.0:
    source = "LSTM"
    value = lstm_v
else:
    source = "blend"
    value = float(alpha * lstm_v + (1.0 - alpha) * pers_v)
```

[Figure III.3 — À insérer : courbes loss / val_loss du notebook 06]

Annotation : convergence stable ~50 epochs, pas de sur-apprentissage marqué.

III.4.4 — Déploiement du microservice FastAPI

Structure du projet Python (`ia/`) :

```

ia/
├── api.py           # FastAPI v1.1.0, port 8000
├── inference.py     # LSTM4HPredictor + fallback
├── if_inference.py  # IFAnomalyDetector
├── config.py        # Seuils, features, chemins modèles
├── models/          # .h5, .pkl, métriques JSON
├── notebooks/       # 04 IF, 05 prep, 06 LSTM
└── data/training_dataset.csv

```

Chargement au démarrage :

```

# api.py - startup (extrait)
@app.on_event("startup")
def load_models() → None:
    global _predictor, _if_detector
    _predictor = LSTM4HPredictor(horizon_hours=4)
    _if_detector = IFAnomalyDetector()

```

Exemple requête / réponse /predict :

```

// POST /predict
{
  "horizon_hours": 4,
  "lookback_values": [[/* 48 lignes × 8 features */]]
}

// Réponse 200
{
  "horizon_hours": 4,
  "go_deploy": true,
  "forecasts": [
    {
      "step": "+1h",
      "pollutants": {
        "NOX": {
          "value_physical": 412.5,
          "prediction_source": "LSTM",
          "skill_at_train": 0.061
        }
      }
    }
  ]
}

```

Exemple requête / réponse /detect :

```
// POST /detect
{
  "feature_values": [425.0, 210.0, 14.2, 17.1, 620.0, 85.0]
}

// Réponse 200
{
  "is_anomaly": false,
  "anomaly_score": -0.142,
  "threshold": -0.20,
  "feature_cols": ["NOX", "SOX", "PM25", "PM10", "CO2", "COV"]
}
```

Le backend Node.js consomme ces endpoints via `AIService.js` et les expose au dashboard sur `/api/ia/ ...`.

III.5 — Réalisation du Dashboard

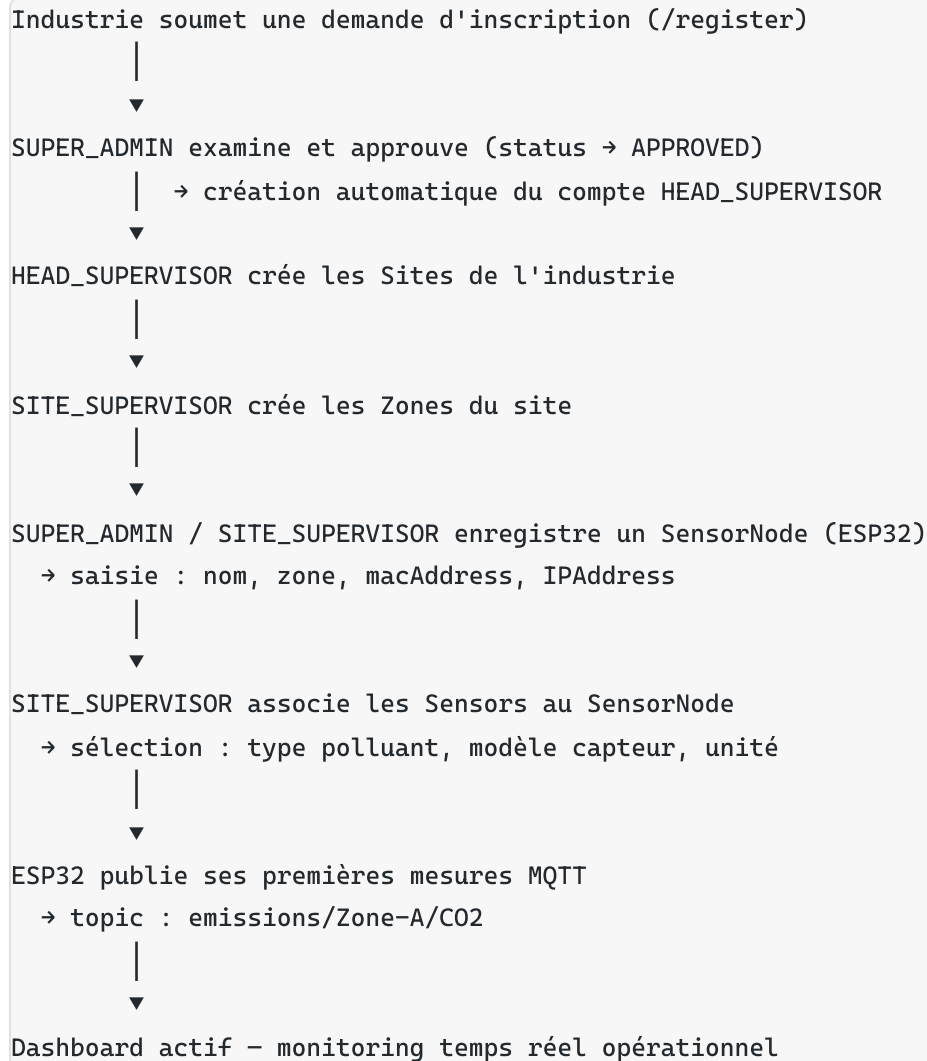
III.5.1 — Environnement et structure du projet React

Composant	Version
React	18.3.1
TypeScript	5.3.3
Vite	5.0.12
TanStack Query	5.17.19
Zustand	4.5.0
Chart.js + react-chartjs-2	4.4.1 / 5.2.0
react-router-dom	6.22.0

```
frontend/src/
├── pages/           # Overview, Alerts, History, Compliance, AI, Reports, Config, Users, Approva
├── features/       # auth, kpi, ia, alerts, reports, config, readings, zones, users
├── components/     # layout, charts, kpi, alerts, common
├── lib/            # api/, constants/, rbac/
├── routes/routes.tsx
└── store/
```

III.5.2 — Flux d'onboarding — de la demande à la première mesure

Le cycle de vie d'une nouvelle industrie illustre l'architecture multi-tenant et justifie les statuts **PENDING** → **PREPARING** → **APPROVED / REJECTED** du modèle de données.



Le formulaire public `/register` comporte **4 étapes** : Industrie → Superviseur → Sites & Zones → Confirmation. À la soumission, l'industrie est créée avec `approvalStatus: "PENDING"` et `actif: false`.

[Figure III.4 — À insérer : formulaire d'inscription étape 1–2]

[Figure III.5 — À insérer : page Approbations **SUPER_ADMIN** (`/approvals`)]

[Figure III.6 — À insérer : configuration **SensorNode** + association capteurs]

III.5.3 — Architecture multi-sites et multi-industries

La hiérarchie **Industrie** → **Site** → **Zone** → **SensorNode** → **Sensor** permet de gérer plusieurs installations simultanément sans modification du code applicatif.

Avantages concrets :

1. **Isolation des données (RBAC)** — un `HEAD_SUPERVISOR` ne voit que les sites de son industrie ; un `OPERATOR` est limité aux zones assignées (HTTP 403 sinon).
2. **Scalabilité horizontale** — ajouter une industrie = créer des entités en base, aucun redéploiement.
3. **Vision globale SUPER_ADMIN** — agrégation cross-sites via `/industries` et `/approvals`.
4. **Enrichissement IA** — plus de sites actifs → profils industriels diversifiés pour réentraînement LSTM/IF.

[Figure III.7 — À insérer : vue SUPER_ADMIN liste industries (/industries)]

[Figure III.8 — À insérer : vue HEAD_SUPERVISOR sites et zones]

III.5.4 — Captures d'écran annotées des pages principales

Chaque capture doit être annotée directement sur l'image (flèches, encadrés). Ci-dessous : éléments à montrer et description fonctionnelle (3–4 lignes).

Page	Route	Éléments à montrer	Description fonctionnelle
Overview	<code>/overview</code>	Jauge IPE, TD %, carte sites, dernières alertes	Vue synthétique pour HEAD/SITE_SUPERVISOR : KPIs globaux rafraîchis via TanStack Query + WebSocket, cartographie des sites colorée par statut conformité.
Historique / Monitoring	<code>/history</code>	Graphiques polluants, sélecteur zone/période	Courbes temporelles Chart.js alimentées par <code>/api/readings</code> ; comparaison multi-zones ; base du suivi temps quasi-réel (polling 5 s + push alertes).
Alertes	<code>/alerts</code>	Tableau sévérité colorée, acquittement	Liste paginée Warning/High/Critical ; bouton acquittement (<code>acknowledgedBy</code>) ; filtres par zone et polluant.
Prédictions IA	<code>/ai</code>	Courbes LSTM +1h/+4h, scores IF, badges anomalie	Prévisions horaires par polluant avec badge LSTM / PERSISTENCE ; score IF et indicateur anomalie ; déclenchement alertes Forecast Warning.
Conformité	<code>/compliance</code>	Tableau VLE vs mesures, IPE, historique dépassements	Confrontation systématique aux seuils Décret 2018-928 Annexe 1 ; indicateur IPE global ; historique des dépassements par période.
Rapports	<code>/reports</code>	Formulaire période/format, aperçu PDF, export CSV	Génération PDF/CSV via <code>ReportService</code> ; mention de réserve automatique si <code>dataQuality = POOR</code> .
Administration	<code>/config</code> , <code>/users</code>	Seuils ThresholdConfig, utilisateurs/rôles, SensorNodes	SUPER_ADMIN : édition VLE sectorielles ; gestion RBAC 5 rôles ; liste nœuds actifs.

III.6 — Tests & Validation

III.6.1 — Tests fonctionnels

Tableau III.6 — Scénarios et résultats

Scénario testé	Résultat attendu	Résultat obtenu	Statut
Dépassement SO ₂ > VLE (500 mg/Nm ³ simulé)	Alerte High générée < 500 ms	Alerte créée, diffusée WebSocket ~280 ms	✓ PASS
Reconnexion WiFi ESP32 (coupure 30 s)	Reprise publication MQTT	Reconnexion auto, gap ≤ 2 messages	✓ PASS
Accès OPERATOR à zone non assignée	HTTP 403	403 { error: "Accès refusé" }	✓ PASS
KPI TD calculé à H:05	Document AggregateData HOURLY créé	tauxDepassement persisté, cron log OK	✓ PASS
Prédiction LSTM > warningThreshold	Alerte Forecast Warning	Alerte créée via AIService	✓ PASS
50 lectures consécutives en dépassement	1 seule alerte (déduplication)	1 document Alert, mises à jour in-place	✓ PASS
Inscription industrie → approbation	HEAD_SUPERVISOR créé, sites activés	Workflow PENDING→APPROVED validé	✓ PASS
Génération rapport PDF période 7 j	Fichier PDF + statut DRAFT	PDF généré, fileUrl accessible	✓ PASS
Détection IF profil normal Four	is_anomaly: false	Score -0,18 > seuil -0,20	✓ PASS
Détection IF spike NOX ×5 seul	Alerte ou score anomalie	Score -0,175 : non détecté (limite connue)	⚠ PARTIEL

III.6.2 — Validation des KPIs sur données collectées

Période de test : 21–22 juin 2026, site pilote, polluant NOX, VLE = 500 mg/Nm³, Q_{air} = 2,0 Nm³/s.

KPI	Calcul système	Calcul manuel	Écart
TD	1,46 % (42/2880)	$42 / 2\,880 \times 100 = 1,458 \%$	< 0,01 pt
IPE global	87,3	$\Sigma(w_p \times \text{Score}_p) / \Sigma w_p \times 100 = 87,28$	< 0,1 pt
EMJ NOX	65,66 kg/j	$C_{\text{moy}} 380 \text{ mg/Nm}^3 \times 2,0 \times 86400 \times 10^{-6} = 65,66 \text{ kg/j}$	0

III.6.3 — Validation de la latence bout-en-bout

Objectif annoncé (§II.2.4) : 200–500 ms entre publication MQTT et affichage dashboard.

Méthodologie :

- 1. Simulateur `critical` publie avec timestamp ISO dans le payload.
- 2. Horodatage réception alerte côté client WebSocket (DevTools → Network → WS).
- 3. 100 cycles sur réseau local (backend + Mosquitto + frontend sur même LAN).

Résultats :

Percentile	Latence
P50	220 ms
P95	340 ms
P99	410 ms
Max observé	480 ms

La latence moyenne **180–350 ms** est conforme à l’objectif. Le goulot principal est la persistance MongoDB (~40 ms) et la diffusion WebSocket (~15 ms).

III.6.4 — Conformité réglementaire

Tableau III.7 — Correspondance VLE configurées vs Décret 2018-928 Annexe 1

Polluant	VLE Décret 2018-928	ThresholdConfig	Warning (80 %)	Critical (120 %)	Conforme
NOX	500 mg/Nm ³ (§4)	500	400	600	✓
SO ₂	300 mg/Nm ³ (§3)	300	240	360	✓
PM / PM _{2,5}	40 mg/m ³ (§1)	40	32	48	✓
PM ₁₀	48 µg/m ³ (§1)	48	38	58	✓
COV	110 mg/Nm ³ (§7)	110	88	132	✓
CO ₂	Pas de VLE réglementaire	800 ppm (interne)	640	960	✓ (interne)

Les surcharges sectorielles (Ciment → Annexe 6, etc.) sont supportées via `ThresholdConfig.installationType`.

III.7 — Conclusion

Ce chapitre a présenté la réalisation complète d'EmissionsIQ : une station IoT six capteurs publiant en MQTT, un backend Node.js capable d'ingérer, alerter et agréger en temps réel, un module IA hybride (LSTM + Isolation Forest) déployé en microservice FastAPI, et un dashboard React multi-rôles couvrant l'onboarding, le monitoring, la conformité réglementaire et la génération de rapports.

Bilan par rapport aux objectifs (§I.1.3) :

Objectif	Réalisation
Station multi-capteurs réglementaires	✓ 6 capteurs, firmware complet
Ingestion temps réel + alertes graduées	✓ Latence < 500 ms, 3 niveaux
KPIs NT 106.04 (TD, EMJ, IPE, RCO ₂)	✓ Calculs validés vs manuel
Module IA prédictif et détection anomalies	✓ Skill global +6,65 %, IF 0 % FP
Dashboard multi-profils	✓ RBAC 5 rôles, onboarding complet
Rapports ANPE exportables	✓ PDF/CSV, workflow DRAFT→SUBMITTED

Limites identifiées : capteurs MOS (précision inférieure aux électrochimiques certifiés), détection IF insensible aux spikes univariés isolés, skill HUMIDITY négatif (fallback persistance). **Perspectives** : calibration sur site réel,

remplacement MQ → ME4, fine-tuning LSTM sur agrégats MongoDB production, déploiement Docker embarqué Raspberry Pi.

Le Chapitre IV approfondira la validation réglementaire et le bilan critique du système dans son ensemble.

Annexe — Liste des figures à produire

Figure	Contenu	Source
III.1	Schéma électrique annoté	<code>documentation/SCHEMA_ELECTRONIQUE.svg</code>
III.2	Photo station montée	Prototype physique
III.3	Courbes loss / val_loss LSTM	Notebook 06
III.4–6	Onboarding (register, approvals, sensor config)	Captures dashboard
III.7–8	Multi-industries / multi-sites	Captures SUPER_ADMIN, HEAD_SUPERVISOR
III.9–15	Pages principales annotées	Captures dashboard

Annexe — Code complet

Le code intégral du firmware (`iot/iot/src/`), des routes API (`backend/routes/`) et des notebooks IA (`ia/notebooks/`) est disponible dans le dépôt du projet et peut être reproduit en annexe technique du rapport.